

## PROFILE

동시성, 이벤트 복구, 실시간 메시징, 과금 정합성을 테스트와 재현 가능한 수치로 검증합니다.

## CORE STACK

Java

Spring Boot

PostgreSQL

Redis

Kafka

WebSocket

Spring Security

JPA

MySQL

k6

## SELECTED WORK

## Concert Booking

GitHub

Ticketing / Reservation · 개인 / BE 1

**문제** 같은 좌석 경합, 대기열 우회, client retry와 Kafka 발행 실패가 한 예매 흐름에서 겹칠 수 있었습니다.

**판단** PostgreSQL을 좌석·예약 기준 데이터로 두고 Queue Token, 좌석 락, Idempotency-Key, Outbox/DLT를 경계별로 분리했습니다.

**결과** 동일 좌석 동시 요청을 1건으로 수렴시키고, Outbox 발행 실패 재시도와 DLT replay의 역등 복구를 통합 테스트로 검증했습니다.

**근거** [시나리오 검증](#) 동일 좌석 경합 10개 동시 요청 → 성공 1, 실패 9, 좌석 상태 HELD 1건

[근거 보기](#)

## Realtime Chat

GitHub

Realtime Messaging · 개인 / BE 1

**문제** WebSocket 연결 성공만으로는 비멤버 구독, room 간 순서 혼선, 수신 누락, 재연결 이후 공백을 설명할 수 없었습니다.

**판단** SUBSCRIBE 단계 인가, roomId Kafka key, 수신자 기준 delivery matrix, afterMessageId sync API로 전달 경계를 나눴습니다.

**결과** 1,000-user 로컬 receiver matrix 3회에서 회차별 99,900건을 중복·누락 없이 검산하고, 구독 인가와 재연결 cursor 경계를 통합 테스트로 고정했습니다.

**근거** [측정 완료](#) 수신자 기준 전달 완전성 1,000 users × 3회 · 회차별 expected 99,900 / unique 99,900 / missing 0 / duplicate 0

[근거 보기](#)

## AI Usage Billing Gateway

GitHub

Billing / Usage Gateway · 개인 / BE 1

**문제** API key 원문 유출, retry에 의한 사용량 중복, provider webhook 재전달, 환불 원장 불균형을 함께 통제해야 했습니다.

**판단** key prefix/hash 저장, request hash 기반 idempotency, providerEventId reserve, append-only reversal ledger로 책임을 분리했습니다.

**결과** 원문 미저장, usage duplicate/conflict, webhook 중복 흡수, 원 거래를 참조하는 balanced refund reversal을 통합 테스트로 검증했습니다.

**근거** [시나리오 검증](#) API Key 저장 경계 raw key는 생성 시 한 번만 반환하고 DB·목록 응답에는 원문을 남기지 않음

[근거 보기](#)

## BorrowMe

GitHub

Rental / Team Project · 11인 팀 프로젝트

**문제** 상품·작성자·해시태그·팔로우 여부를 조립하는 목록 경로와 동시 예약 재고를 회귀 없이 유지해야 했습니다.

**판단** fetch join과 bulk follow lookup에 query-count guard를 두고, 현재 조건을 기록한 k6 snapshot을 과거 기록과 분리했습니다.

**결과** 현재 상품 목록 snapshot에서 p95 358.1088ms와 HTTP failure 0을 기록했고, 핵심 조회 경로의 SQL 상한과 예약 재고 정합성을 Testcontainers로 검증했습니다.

**근거** [측정 완료](#) 상품 목록 현재 snapshot p95 358.1088ms · HTTP failure 0 · checks 10,683/10,683

[근거 보기](#)